

# DDS 상호운용성 Wire Protocol™ (DDSI-RTPS™) 사양

## 출처 (Reference)

- 발행 기관: Object Management Group (OMG)
  - 문서명: DDS Interoperability Wire Protocol™ (DDSI-RTPS™) Specification, Version 2.5
  - 문서 번호: formal/22-04-01
  - 발행일: 2022년 4월
  - URL: <https://www.omg.org/spec/DDSI-RTPS/2.5/PDF>
- 

## 1. 범위 (Scope)

이 명세서(Specification)는 데이터 분산 서비스(DDS: Data Distribution Service) 구현체 간의 상호 운용성을 허용하는 유선 프로토콜(Wire Protocol)을 정의합니다.

DDS 명세서의 주요 동기는 애플리케이션의 이식성(Portability)을 보장하는 것이었습니다. 이 문서는 "네트워크 상의 상호 운용성(Interoperability on the network)" 을 보장하기 위해 RTPS(Real-Time Publish-Subscribe) 프로토콜을 정의합니다.

이 유선 프로토콜의 주요 목표는 다음과 같습니다:

1. **DDS 애플리케이션 인터페이스 사양을 기반으로 하는 다른 구현체 간의 상호 운용성 보장.**
  - 이는 서로 다른 벤더가 구현한 DDS를 사용하는 애플리케이션들이 데이터를 교환할 수 있음을 의미합니다.
2. **원하는 서비스 품질(QoS)과 속성(Attribute)을 지원.**
  - DDS 명세서에 정의된 QoS(예: 신뢰성, 지속성 등)를 네트워크상에서 구현합니다.

이 문서는 다음과 같은 내용을 포함합니다:

- RTPS 프로토콜의 메시지 형식(Message Formats)
- RTPS 프로토콜의 동작(Behavior) 및 상태 머신(State Machine)
- DDS 엔티티(Entity)를 자동으로 발견하기 위한 검색(Discovery) 프로토콜

- DDS Security(보안) 사양 지원을 위한 확장
- 

## 2. 적합성 (Conformance)

이 사양에 대한 적합성을 주장하는 구현체는 다음 요구사항을 준수해야 합니다.

### 1. 플랫폼 독립 모델(PIM) 준수:

- 제8장(Message Module)과 제9장(Behavior Module)에 정의된 RTPS PIM의 구조 및 동작을 준수해야 합니다.

### 2. 플랫폼 고유 모델(PSM) 준수:

- 제10장(Data Encapsulation)에 정의된 데이터 캡슐화 방식을 지원해야 합니다. 기본적으로 CDR(Common Data Representation)을 지원해야 합니다.

### 3. 검색 프로토콜 준수:

- 제8.5절에 정의된 단순 검색 프로토콜(SPDP 및 SEDP)을 구현하여 상호 운용성을 보장해야 합니다.

### 4. DDS 보안 사양 준수 (선택적):

- 보안 기능을 제공하는 경우, DDS Security 사양과 연계된 RTPS 확장 필드를 올바르게 처리해야 합니다.

구현체는 'DDS Interoperability Wire Protocol'을 준수한다고 명시해야 하며, 지원하는 프로토콜 버전을 명확히 밝혀야 합니다.

---

## 3. 규범적 참조 (Normative References)

다음 문서는 이 문서의 적용을 위해 필수적입니다. 날짜가 명기된 참조의 경우 해당 버전만 적용됩니다.

- [DDS]: Object Management Group, *Data Distribution Service (DDS) for Real-time Systems Specification*, Version 1.4.
- [DDS-SECURITY]: Object Management Group, *DDS Security Specification*, Version 1.1.
- [CDR]: Object Management Group, *Common Object Request Broker Architecture (CORBA) Core Specification*, Part 2: CDR Transfer Syntax.

- [RFC 768]: IETF RFC 768, *User Datagram Protocol (UDP)*.
  - [RFC 791]: IETF RFC 791, *Internet Protocol (IP)*.
- 

## 4. 용어 및 정의 (Terms and Definitions)

이 문서의 목적을 위해 다음의 용어와 정의가 적용됩니다.

- **바이트(Byte)**: 8비트의 데이터 단위 (Octet과 동일).
  - **옥텟(Octet)**: 8비트로 구성된 데이터 단위. 이 문서에서는 바이트 대신 명확성을 위해 옥텟이라는 용어를 주로 사용합니다.
  - **CDR (Common Data Representation)**: OMG에서 정의한 데이터 직렬화(Serialization) 표준 형식. RTPS 패킷 내의 데이터 페이로드는 CDR 형식을 따릅니다.
  - **GUID (Globally Unique Identifier)**: RTPS 엔티티(Entity)를 전역적으로 고유하게 식별하는 ID입니다.
  - **참가자(Participant)**: DDS 도메인에 참여하는 최상위 레벨의 컨테이너 엔티티입니다.
  - **엔드포인트(Endpoint)**: 데이터를 실제로 송수신하는 단위로, RTPS Writer와 RTPS Reader를 통칭합니다.
  - **RTPS Writer**: 데이터를 네트워크로 전송하는 엔드포인트입니다.
  - **RTPS Reader**: 네트워크로부터 데이터를 수신하는 엔드포인트입니다.
  - **서브메시지(Submessage)**: RTPS 메시지를 구성하는 기본 단위입니다. 하나의 RTPS 메시지는 하나 이상의 서브메시지로 구성됩니다.
- 

## 5. 심볼 및 약어 (Symbols)

이 문서에서 사용되는 주요 약어는 다음과 같습니다.

- **DDS**: Data Distribution Service
- **RTPS**: Real-Time Publish-Subscribe
- **PIM**: Platform Independent Model (플랫폼 독립 모델)
- **PSM**: Platform Specific Model (플랫폼 고유 모델)
- **GUID**: Globally Unique Identifier

- **QoS:** Quality of Service
  - **CDR:** Common Data Representation
  - **UDP:** User Datagram Protocol
  - **IP:** Internet Protocol
  - **SPDP:** Simple Participant Discovery Protocol (단순 참가자 검색 프로토콜)
  - **SEDP:** Simple Endpoint Discovery Protocol (단순 엔드포인트 검색 프로토콜)
- 

## 6. 추가 정보 (Additional Information)

### 6.1 문서 구성 (Document Structure)

이 문서는 다음과 같이 구성되어 있습니다.

- **제1장 ~ 제6장:** 범위, 적합성, 참조, 용어, 약어 및 이 추가 정보 섹션을 포함한 서론입니다.
  - **제7장 (RTPS PIM 개요):** RTPS 프로토콜의 전반적인 구조와 모델을 설명합니다. (가장 중요한 개념적 설명을 포함)
  - **제8장 (PIM - 메시지 모듈):** RTPS 메시지의 구조, 헤더 포맷, 각 서브메시지(DATA, HEARTBEAT, ACKNACK 등)의 비트 단위 형식을 정의합니다.
  - **제9장 (PIM - 동작 모듈):** Writer와 Reader 간의 통신 시퀀스와 상태 머신(State Machine)을 정의합니다. 데이터 전송, 손실 감지, 재전송 로직이 여기에 포함됩니다.
  - **제10장 (PIM - 검색 모듈):** 네트워크 상에서 다른 참가자를 찾고 연결을 맺는 Discovery 과정을 정의합니다.
- 

## 7. RTPS 플랫폼 독립 모델 (PIM)

RTPS(Real-Time Publish-Subscribe) 프로토콜은 **플랫폼 독립 모델(PIM: Platform Independent Model)**로 기술됩니다. 이는 프로토콜의 동작과 구조를 특정 운영체제나 프로그래밍 언어에 종속되지 않는 방식으로 정의함을 의미합니다.

RTPS PIM은 4개의 주요 모듈로 구성됩니다:

1. **구조 모듈 (Structure Module):** 통신 엔드포인트(Endpoint)와 그들의 관계를 정의합니다.

2. **메시지 모듈 (Messages Module):** 엔드포인트 간에 교환되는 메시지의 형식을 정의합니다.
3. **동작 모듈 (Behavior Module):** 엔드포인트 간의 올바른 상호작용 및 상태 전이를 정의합니다.
4. **검색 모듈 (Discovery Module):** 엔티티들이 서로를 자동으로 발견하고 구성하는 방법을 정의합니다.

## 7.1 구조 모듈 (Structure Module)

구조 모듈은 RTPS 프로토콜에 참여하는 엔티티(Entity)들을 정의합니다. 이 엔티티들은 DDS 사양에 정의된 엔티티들과 일대일로 매핑됩니다.

### 7.1.1 RTPS 엔티티 (RTPS Entity)

모든 RTPS 엔티티의 기본 클래스입니다.

- **GUID (Globally Unique Identifier):** 모든 RTPS 엔티티는 도메인 내에서 유일한 식별자인 GUID를 가집니다. GUID를 통해 타겟 엔티티를 식별하고 메시지를 라우팅합니다.

### 7.1.2 RTPS 참가자 (RTPS Participant)

DDS의 `DomainParticipant`에 대응됩니다.

- **역할:** Writer와 Reader를 포함하는 컨테이너 역할을 합니다.
- **속성:**
  - `defaultUnicastLocatorList`: 별도로 지정되지 않은 경우, 이 참가자 내의 엔드포인트들이 수신 대기하는 기본 주소 목록입니다.
  - `protocolVersion`: 사용 중인 RTPS 프로토콜의 버전입니다.
  - `vendorId`: 구현 벤더를 식별하는 ID입니다.

### 7.1.3 RTPS 엔드포인트 (RTPS Endpoint)

RTPS Writer와 RTPS Reader의 공통 부모 클래스입니다.

- **속성:**
  - `topicKind`: 엔드포인트가 다루는 데이터의 종류(No Key 또는 With Key)를 나타냅니다.
  - `reliabilityLevel`: 신뢰성 수준(Best-Effort 또는 Reliable)을 나타냅니다.
  - `unicastLocatorList`: 유니캐스트 데이터를 수신할 주소 목록입니다.
  - `multicastLocatorList`: 멀티캐스트 데이터를 수신할 주소 목록입니다.

### 7.1.4 RTPS Writer

DDS의 **DataWriter**에 대응됩니다.

- **역할:** 데이터의 변경 사항(CacheChange)을 네트워크상의 일치하는 RTPS Reader들에게 전송합니다.
- **동작:**
  - DDS DataWriter로부터 데이터를 받아 **WriterHistoryCache**에 저장합니다.
  - RTPS 프로토콜을 통해 이 변경 사항들을 리모트 Reader들에게 전송합니다.
- **주요 속성:**
  - **pushMode:** true일 경우 데이터를 주도적으로 전송하며, false일 경우 Reader의 요청이 있을 때만 전송합니다.
  - **heartbeatPeriod:** Reliable 통신 시, 생존 신호(Heartbeat)를 보내는 주기입니다.

### 7.1.5 RTPS Reader

DDS의 **DataReader**에 대응됩니다.

- **역할:** 네트워크상의 RTPS Writer로부터 데이터 변경 사항을 수신합니다.
- **동작:**
  - 수신한 데이터를 **ReaderHistoryCache**에 저장합니다.
  - DDS DataReader가 이 데이터를 애플리케이션에 전달할 수 있도록 합니다.
- **주요 속성:**
  - **expectsInlineQos:** 데이터 서브메시지 내에 QoS(품질 정책)가 포함되기를 기대하는지 여부입니다.

---

## 7.2 히스토리 캐시 (HistoryCache)

히스토리 캐시는 DDS와 RTPS 간의 데이터 흐름을 연결하는 핵심 인터페이스입니다. RTPS Writer와 Reader는 각각 자신의 로컬 저장소인 **HistoryCache**를 가집니다.

- **WriterHistoryCache:**
  - DDS DataWriter가 데이터를 쓸(Write) 때마다 **CacheChange**가 생성되어 이곳에 추가됩니다.
  - RTPS Writer는 이곳에 있는 변경 사항들을 네트워크로 전송합니다.
- **ReaderHistoryCache:**

- RTPS Reader가 네트워크로부터 데이터를 받으면 **CacheChange**를 생성하여 이곳에 추가합니다.
- DDS DataReader는 이곳에 저장된 데이터를 읽어 애플리케이션에 전달합니다.

### 7.2.1 캐시 변경 (CacheChange)

히스토리 캐시에 저장되는 데이터의 기본 단위입니다. 각 **CacheChange**는 다음 정보를 포함합니다.

- **Kind**: 변경의 종류 (ALIVE, NOT\_ALIVE\_DISPOSED, NOT\_ALIVE\_UNREGISTERED 등).
- **WriterGUID**: 이 변경을 생성한 Writer의 GUID.
- **SequenceNumber**: Writer가 부여한 고유한 순련 번호(순서 보장용).
- **Data**: 실제 사용자 데이터(Payload).

## 7.3 RTPS 엔티티와 DDS 엔티티의 관계

RTPS 프로토콜은 DDS 사양과 독립적일 수 있도록 설계되었지만, 일반적으로 다음과 같이 매핑됩니다.

| RTPS Entity      | DDS Entity        | 비고                   |
|------------------|-------------------|----------------------|
| RTPS Participant | DomainParticipant | 통신의 주체이자 컨테이너        |
| RTPS Writer      | DataWriter        | 특정 Topic에 데이터를 쓰는 주체 |
| RTPS Reader      | DataReader        | 특정 Topic의 데이터를 읽는 주체 |
| HistoryCache     | (Conceptual)      | DDS의 Data Cache와 유사함 |

### [번역자 주석]

- **Locator(로케이터)**: 네트워크 주소(IP)와 포트(Port) 정보를 포함하는 전송 계층의 주소 개념입니다.
- **SequenceNumber**: RTPS는 이 번호를 사용하여 패킷의 순서를 맞추고 손실된 패킷(Gap)을 감지합니다. 64비트 정수로 정의됩니다.

## 8. 메시지 모듈 (Message Module)

메시지 모듈은 RTPS Writer와 RTPS Reader 간에 정보를 교환하기 위한 원자적 단위인 **\*\*메시지(Message)\*\***의 구조와 내용을 정의합니다.

## 8.1 개요

RTPS 프로토콜은 패킷 단위로 통신하며, 하나의 **RTPS 메시지**는 다음과 같이 구성됩니다.

1. **고정 크기의 헤더 (Header)**: 메시지의 시작을 알리고 버전을 식별합니다.
2. **서브메시지 (Submessages)의 나열**: 실제 정보를 담고 있는 가변 길이의 블록들입니다.

## 8.2 기본 구조 및 규칙

### 8.2.1 정렬 (Alignment) 및 엔디안(Endianness)

- **정렬**: RTPS 메시지 내의 모든 서브메시지는 **32비트(4바이트) 경계**에 맞춰 정렬됩니다.
- **엔디안**: RTPS는 빅 엔디안(Big Endian)과 리틀 엔디안(Little Endian)을 모두 지원합니다. 각 서브메시지의 헤더에는 해당 서브메시지가 어떤 엔디안으로 인코딩되었는지를 나타내는 **\*\*Endianness Flag (E)\*\***가 포함되어 있습니다.

---

## 8.3 메시지 헤더 (Message Header)

모든 RTPS 메시지는 반드시 **Header**로 시작합니다. 헤더는 수신자가 메시지가 유효한 RTPS 프로토콜인지 확인하고, 누가 보냈는지 식별하게 해줍니다.

**헤더 구조 (총 16바이트)**:

1. **ProtocolId (4바이트)**: '**R**', '**T**', '**P**', '**S**'라는 아스키 문자가 들어갑니다.
2. **ProtocolVersion (2바이트)**: 메이저 버전과 마이너 버전 (예: 2.5).
3. **VendorId (2바이트)**: RTPS를 구현한 벤더의 식별자입니다. (예: RTI, eProsima 등)
4. **GuidPrefix (12바이트)**: 메시지를 전송한 **Participant**의 **고유 식별자 앞부분**입니다. 이를 통해 수신자는 이 메시지가 어떤 기기/프로세스에서 왔는지 알 수 있습니다.

---

## 8.4 서브메시지 (Submessage) 구조



헤더 뒤에는 하나 이상의 서브메시지가 옵니다. 모든 서브메시지는 **SubmessageHeader**로 시작합니다.

서브메시지 헤더 구조:

1. **SubmessageId (1바이트)**: 서브메시지의 종류를 나타냅니다. (예: DATA=0x15, HEARTBEAT=0x07)
  2. **Flags (1바이트)**: 서브메시지 처리 옵션을 나타내는 비트 플래그입니다.
    - **E (Endianness)**: 1이면 Little Endian, 0이면 Big Endian.
  3. **SubmessageLength (2바이트)**: 이어지는 내용(Content)의 길이를 나타냅니다.
- 

## 8.5 서브메시지의 분류 (Types of Submessages)

서브메시지는 기능에 따라 크게 두 가지로 분류됩니다.

### 1. 엔티티 서브메시지 (Entity Submessages)

특정 Writer나 Reader를 대상으로 하는 메시지입니다.

- **DATA**: 애플리케이션 데이터를 전송합니다.
- **DATA\_FRAG**: 데이터가 너무 클 경우 조각내어(Fragment) 전송합니다.
- **HEARTBEAT**: Writer가 자신의 생존을 알리고, Reader에게 상태 확인을 요청합니다.
- **ACKNACK**: Reader가 Writer에게 데이터 수신 여부(긍정/부정 응답)를 보냅니다.
- **GAP**: 더 이상 유효하지 않은 데이터 시퀀스 번호를 건너뛰라고 알립니다.

### 2. 인터프리터 서브메시지 (Interpreter Submessages)

메시지 처리 문맥(Context)을 수정하는 메시지입니다. 다음에 오는 서브메시지들에 영향을 줍니다.

- **INFO\_TS (Timestamp)**: 다음에 오는 서브메시지들이 언제 생성되었는지 타임스탬프를 제공합니다.
  - **INFO\_DST (Destination)**: 메시지의 목적지 GuidPrefix를 재설정합니다.
  - **INFO\_SRC (Source)**: 메시지의 출발지 정보를 재설정합니다.
- 

## 8.6 주요 서브메시지 상세 정의

가장 빈번하게 사용되는 핵심 서브메시지들의 상세 내용입니다.

### 8.6.1 DATA Submessage

DDS **DataWriter**에서 **DataReader**로 데이터 변경 사항을 전달합니다.

- **구성:**
  - **ExtraFlags:** 추가 플래그.
  - **OctetsToInlineQos:** 실제 데이터 앞에 오는 QoS 파라미터의 길이.
  - **ReaderId / WriterId:** 송수신 엔드포인트 ID.
  - **WriterSN:** 시퀀스 번호 (순서 보장).
  - **InlineQos** (옵션): 데이터와 함께 전송되는 QoS 정책.
  - **SerializedPayload:** 실제 사용자 데이터.

### 8.6.2 HEARTBEAT Submessage (신뢰성 통신용)

Writer가 Reader에게 "내가 여기까지 보냈다"고 알리는 신호입니다.

- **역할:** Reader가 놓친 데이터가 있는지 확인하게 유도합니다.
- **구성:**
  - **ReaderId / WriterId.**
  - **FirstSN:** Writer가 캐시에 가지고 있는 첫 번째 시퀀스 번호.
  - **LastSN:** Writer가 캐시에 가지고 있는 마지막 시퀀스 번호.
  - **Count:** 하트비트 카운터 (중복 처리 방지).

### 8.6.3 ACKNACK Submessage (신뢰성 통신용)

Reader가 Writer에게 보내는 응답입니다.

- **역할:** "데이터 잘 받았다" 혹은 "몇 번 데이터가 빠졌다, 다시 보내달라"고 요청합니다.
- **구성:**
  - **ReaderId / WriterId.**
  - **ReaderSNState:** 비트맵(Bitmap) 형식을 사용하여 누락된 시퀀스 번호들을 효율적으로 표시합니다.
  - **Count:** AckNack 카운터.

### 8.6.4 INFO\_TS (Timestamp)

- **역할:** 이 서브메시지 뒤에 오는 서브메시지들이 발생한 **절대 시간**을 지정합니다.
  - **특징:** 별도의 데이터 페이로드 없이 시간 정보만 포함하며, 일반적으로 DATA 서브메시지 바로 앞에 위치합니다.
- 

## [번역자 주석]

제8장은 구현(Implementation) 관점에서 가장 중요한 장입니다. 패킷 캡처 도구(Wireshark 등)로 RTPS 패킷을 뜯어볼 때 보이는 내용이 바로 이 장에 정의된 구조입니다.

- **Flags의 중요성:** 각 서브메시지의 헤더에 있는 **Flags** 필드는 매우 중요합니다. 예를 들어, DATA 서브메시지에서 **D** 플래그가 1이면 'Data'가 포함되어 있다는 뜻이고, **K** 플래그가 1이면 데이터 대신 'Key'(식별자)만 포함되어 있다는 뜻(예: 인스턴스 삭제 시)입니다.
- 

## 9. 동작 모듈 (Behavior Module)

동작 모듈은 RTPS 엔티티(Writer, Reader)들이 시간의 흐름에 따라 서로 메시지를 교환하고 내부 상태를 변경하는 규칙을 정의합니다. 이 장은 RTPS 프로토콜의 가장 핵심적인 부분으로, 앞서 정의한 구조(7장)와 메시지(8장)를 사용하여 Writer와 Reader가 어떻게 데이터를 주고받고, 손실을 복구하는지에 대한 동적인 절차(Sequence)와 상태 머신(State Machine)을 정의합니다.

### 9.1 개요 및 통신 모드

RTPS 프로토콜은 상호 운용성과 확장성을 위해 두 가지 주요 참조 구현(Reference Implementation)을 정의합니다.

#### 1. 비상태 (Stateless) 모드

- **특징:** 엔드포인트가 연결된 원격 엔드포인트의 상태를 기억하지 않습니다.
- **용도:**
  - **Best-Effort** 통신 (데이터 손실 허용).
  - **Discovery** 단계 (누가 있는지 모르는 상태에서 멀티캐스트 전송).
  - 메모리가 극도로 제한된 환경.

## 2. 상태 (Stateful) 모드

- **특징:** 엔드포인트가 연결된 모든 원격 엔드포인트의 상태를 추적하고 유지합니다.
  - **용도:** **Reliable** 통신 (데이터 손실 없는 전송).
  - **동작:** Writer는 각 Reader가 어디까지 데이터를 받았는지 알고 있으며, Reader는 Writer가 보낸 데이터 중 무엇이 빠졌는지 추적합니다.
- 

## 9.2 RTPS Writer의 동작

Writer는 데이터를 생성하고 네트워크로 전송하는 주체입니다.

### 9.2.1 StatelessWriter

- **동작:** **WriterHistoryCache**에 데이터가 추가되면, 미리 설정된 모든 수신자(Locators)에게 데이터를 전송합니다.
- **피드백:** 수신자로부터 어떤 응답(ACK)도 기다리지 않습니다. 단순히 "쓰고 잊어버리는(Fire-and-forget)" 방식입니다.

### 9.2.2 StatefulWriter (신뢰성 통신의 핵심)

Reliable 통신을 지원하기 위해 **StatefulWriter**는 연결된 각 Reader마다 **ReaderProxy**라는 객체를 생성하여 관리합니다.

- **ReaderProxy의 역할:**
  1. 특정 원격 Reader에 대한 정보를 유지합니다.
  2. **중요 속성:** **acknowledged\_changes** (해당 Reader가 수신 확인한 시퀀스 번호의 목록).
- **데이터 전송 절차:**
  1. **전송:** 캐시에 변경 사항이 생기면 DATA 서브메시지를 전송합니다.
  2. **Heartbeat:** 주기적으로 혹은 데이터 전송 직후에 **HEARTBEAT** 서브메시지를 보내, 자신의 상태(FirstSN, LastSN)를 알립니다.
  3. **AckNack 처리:** Reader로부터 **ACKNACK**를 받으면:
    - Reader가 받지 못했다고 보고한(NACK) 데이터들을 찾습니다.
    - 해당 데이터들을 **\*\*재전송(Repair)\*\***합니다.
    - Reader가 받았다고 보고한 시퀀스 번호까지 상태를 업데이트합니다.

---

## 9.3 RTPS Reader의 동작

Reader는 데이터를 수신하고, 필요시 재전송을 요청하는 주체입니다.

### 9.3.1 StatelessReader

- **동작:** 들어오는 모든 데이터를 조건 없이 수락하여 처리합니다.
- **피드백:** Writer에게 어떠한 정보도 다시 보내지 않습니다.

### 9.3.2 StatefulReader (신뢰성 통신의 핵심)

Reliable 통신을 위해 **StatefulReader**는 매칭된 각 Writer마다 **WriterProxy**라는 객체를 생성하여 관리합니다.

- **WriterProxy의 역할:**
  1. 특정 원격 Writer에 대한 상태를 유지합니다.
  2. **중요 속성:** **available\_changes** (해당 Writer로부터 받을 수 있는 유효한 시퀀스 번호의 범위), **missing\_changes** (빠진 데이터 목록).
- **데이터 수신 및 복구 절차:**
  1. **DATA 수신:** 데이터가 도착하면 시퀀스 번호를 확인합니다. 순서에 맞으면 처리하고, 순서가 비어 있으면(Gap) '누락'으로 표시합니다.
  2. **HEARTBEAT 처리:** Writer로부터 HEARTBEAT를 받으면:
    - "Writer는 10번까지 보냈다는데, 나는 8번이 없다"는 식으로 누락을 감지합니다.
    - 누락된 데이터가 있으면 **ACKNACK** 메시지를 작성하여 Writer에게 보냅니다.
  3. **재전송 대기:** 요청한 데이터가 올 때까지 기다립니다.

---

## 9.4 신뢰성 프로토콜 (Reliability Protocol) 요약

RTPS의 신뢰성 메커니즘은 TCP와 다릅니다. TCP는 긍정 확인(Positive ACK) 위주이지만, RTPS는 **부정 확인(Negative ACK)** 위주입니다.

1. **Writer:** 데이터를 쏟아냅니다(Push).

2. **Writer**: "나 1번부터 100번까지 보냈어, 다 받았니?" 라고 묻습니다 (**Heartbeat**).
3. **Reader (데이터를 다 받은 경우)**: 아무 말도 하지 않습니다 (침묵은 긍정). ※ 설정에 따라 긍정 *ACK*를 보낼 수도 있습니다.
4. **Reader (데이터가 빠진 경우)**: "95번이 안 왔어, 다시 줘" 라고 말합니다 (**AckNack**).
5. **Writer**: 95번만 다시 보냅니다 (**Repair Data**).

이 방식은 멀티캐스트 환경에서 네트워크 트래픽을 획기적으로 줄여줍니다 (수백 개의 Reader가 일일이 "잘 받았다"고 답하지 않아도 되기 때문).

---

## [번역자 주석] 기술적 이해를 돕기 위한 팁

1. **Proxy(프록시) 개념**:
    - 9장에서 계속 나오는 **ReaderProxy**와 **WriterProxy**는 실제 네트워크 너머에 있는 엔티티가 아닙니다. 내 메모리 안에 있는 "상대방의 분신(그림자)"입니다.
    - **Writer**는 자신의 메모리에 있는 **ReaderProxy**를 보고 "아, 저 녀석은 아직 5번 데이터를 못 받았구나"라고 판단합니다.
  2. **Push vs Pull**:
    - 데이터 전송은 **Push** (**Writer**가 밀) 방식입니다.
    - 데이터 복구(재전송)는 **Pull** (**Reader**가 요청) 방식에 가깝습니다.
- 

## 10. 검색 모듈 (Discovery Module)

검색 모듈은 RTPS 참가자(Participant)들이 네트워크 상에서 서로의 존재를 인식하고, 각자가 가진 엔드포인트(Writer/Reader) 정보를 교환하여 통신 경로를 수립하는 프로토콜을 정의합니다.

RTPS의 검색 과정은 크게 두 단계로 나뉩니다.

1. **PDP (Participant Discovery Protocol)**: 참가자(기기/프로세스)끼리 서로를 발견하는 단계.
2. **EDP (Endpoint Discovery Protocol)**: 발견된 참가자 내부의 **Writer**와 **Reader** 정보를 교환하는 단계.

### 10.1 단순 참가자 검색 프로토콜 (SPDP: Simple Participant Discovery Protocol)

SPDP는 네트워크에 새로운 참가자가 들어왔음을 알리고 감지하는 기본 프로토콜입니다.

- **동작 방식:**
  - 모든 참가자는 **SPDP Writer**와 **SPDP Reader**를 기본적으로 가집니다.
  - SPDP Writer는 주기적으로(Announce Period) 자신의 정보를 담은 메시지(**SPDPdiscoveredParticipantData**)를 **멀티캐스트(Multicast)** 주소로 전송합니다.
- **교환 정보:**
  - 자신의 GUID Prefix.
  - 유니캐스트/멀티캐스트 수신 주소(Locator List).
  - 프로토콜 버전 및 벤더 ID.
- **결과:**
  - SPDP 메시지를 수신한 참가자는 상대방을 인식하고, 상대방과 통신하기 위한 **\*\*참가자 프록시(ParticipantProxy)\*\***를 생성합니다. 이것이 통신의 첫 단추입니다.

## 10.2 단순 엔드포인트 검색 프로토콜 (SEDP: Simple Endpoint Discovery Protocol)

참가자끼리 서로를 인식(SPDP 완료)했다면, 이제 구체적으로 어떤 데이터를 주고받을지(Topic)에 대한 정보를 교환해야 합니다. 이를 위해 SEDP를 사용합니다.

- **동작 방식:**
  1. SPDP를 통해 맺어진 연결을 바탕으로, 이번에는 **신뢰성 있는(Reliable)** 통신을 사용하여 정보를 교환합니다.
  2. RTPS 표준은 이 정보를 교환하기 위해 **\*\*사전 정의된(Pre-defined)** 4개의 내장 토픽(Built-in Topics)\*\*을 사용합니다.
- **내장 엔드포인트 (Built-in Endpoints):**
  1. **Publications Writer/Reader:** "나 이런 데이터를 쓸(Write) 거야"라고 알림 (Topic 이름, Type 이름, QoS 등).
  2. **Subscriptions Writer/Reader:** "나 이런 데이터를 읽고(Read) 싶어"라고 알림.

## 10.3 검색 트래픽의 내용 (Discovery Traffic)

검색 단계에서 오고 가는 데이터는 일반 데이터와 구조가 다릅니다. 확장성을 위해 **ParameterList** 형식을 사용합니다.

### 10.3.1 파라미터 리스트 (ParameterList)

- **구조:** **PID (Parameter ID)** + **Length** + **Value** 의 연속된 구조입니다. (TLV 방식과 유사)

- **장점:**
  - 새로운 QoS 정책이 추가되더라도, 이전 버전의 구현체는 모르는 PID를 건너뛰고 해석할 수 있어 하위 호환성이 보장됩니다.
  - 마지막은 항상 **PID\_SENTINEL**로 끝납니다.

### 10.3.2 주요 파라미터 (Key Parameters)

- **PID\_TOPIC\_NAME:** 데이터의 주제 이름 (예: "RobotSensor").
- **PID\_TYPE\_NAME:** 데이터 타입 이름 (예: "SensorDataStruct").
- **PID\_RELIABILITY:** 신뢰성 QoS 설정 (Reliable 또는 Best-Effort).
- **PID\_DURABILITY:** 지속성 QoS 설정 (Volatile 또는 Transient-Local).

---

## 10.4 매칭 및 연결 (Matching and Connection)

SEDP를 통해 상대방의 Writer/Reader 정보를 수신하면, 각 엔티티는 **매칭(Matching)** 과정을 수행합니다.

1. **토픽 일치:** Topic 이름과 Type 이름이 같은가?
2. **QoS 호환성(RxO: Requested vs Offered):**
  - Writer가 제공하는 품질(Offered)이 Reader가 요구하는 품질(Requested)보다 높거나 같아야 합니다.
  - 예: Reader가 "Reliable"을 원하는데 Writer가 "Best-Effort"만 지원하면 연결되지 않습니다.
3. **결과:** 모든 조건이 맞으면 논리적 연결이 성립되고, 이때부터 애플리케이션 데이터(User Data)가 흐르기 시작합니다.

---

## 10.5 내장 엔티티 ID (Built-in Entity Ids)

RTPS 헤더 내에서 검색 트래픽을 구분하기 위해 고정된 EntityID를 사용합니다. (매우 중요)

| Entity ID  | 용도              | 설명                |
|------------|-----------------|-------------------|
| 0x000100c2 | SPDP Writer     | 참가자 정보 송신용        |
| 0x000100c7 | SPDP Reader     | 참가자 정보 수신용        |
| 0x000300c2 | SEDP Pub Writer | 자신의 Writer 정보 송신용 |



|            |                 |                   |
|------------|-----------------|-------------------|
| 0x000300c7 | SEDP Pub Reader | 상대의 Writer 정보 수신용 |
| 0x000400c2 | SEDP Sub Writer | 자신의 Reader 정보 송신용 |
| 0x000400c7 | SEDP Sub Reader | 상대의 Reader 정보 수신용 |

(참고: 마지막 바이트 **c2**는 Writer, **c7**은 Reader를 의미하는 RTPS 관례입니다.)

## [번역자 주석] 10장의 핵심 요약

### 1. 부트스트래핑(Bootstrapping):

- SPDP는 “답이 먼저냐 알이 먼저냐” 문제를 해결합니다. 아무것도 모르는 상태에서 멀티캐스트로 “나 여기 있어!”라고 외치는 것이 시작입니다.

### 2. 시스템 트래픽 vs 사용자 트래픽:

- 10장에서 설명하는 모든 통신은 사용자가 보내는 데이터가 아니라, 미들웨어 자체가 돌아가기 위한 \*\*메타 데이터(Meta-data)\*\*입니다.
- 따라서 네트워크 패킷을 분석할 때, **EntityID**를 보고 이것이 ‘데이터’인지 ‘검색 정보’인지 구분해야 합니다.

### 3. QoS 매칭:

- DDS가 강력한 이유입니다. 단순히 이름만 같다고 연결하는 것이 아니라, “나는 1초에 한 번은 꼭 받아야 해(Deadline)” 같은 조건이 맞아떨어지면 연결해 줍니다. 이 로직이 SEDP 과정에 포함되어 있습니다.

## 10. 검색 모듈 (Discovery Module)

검색 모듈은 RTPS 참가자(Participant)들이 네트워크 상에서 서로의 존재를 인식하고, 각자가 가진 엔드포인트(Writer/Reader) 정보를 교환하여 통신 경로를 수립하는 프로토콜을 정의합니다.

RTPS의 검색 과정은 크게 두 단계로 나뉩니다.

1. **PDP (Participant Discovery Protocol)**: 참가자(기기/프로세스)끼리 서로를 발견하는 단계.
2. **EDP (Endpoint Discovery Protocol)**: 발견된 참가자 내부의 Writer와 Reader 정보를 교환하는 단계.

### 10.1 단순 참가자 검색 프로토콜 (SPDP: Simple Participant Discovery Protocol)

SPDP는 네트워크에 새로운 참가자가 들어왔음을 알리고 감지하는 기본 프로토콜입니다.

- **동작 방식:**
  - 모든 참가자는 **SPDP Writer**와 **SPDP Reader**를 기본적으로 가집니다.
  - SPDP Writer는 주기적으로(Announce Period) 자신의 정보를 담은 메시지(**SPDPdiscoveredParticipantData**)를 **멀티캐스트(Multicast)** 주소로 전송합니다.
- **교환 정보:**
  - 자신의 GUID Prefix.
  - 유니캐스트/멀티캐스트 수신 주소(Locator List).
  - 프로토콜 버전 및 벤더 ID.
- **결과:**
  - SPDP 메시지를 수신한 참가자는 상대방을 인식하고, 상대방과 통신하기 위한 **\*\*참가자 프록시(ParticipantProxy)\*\***를 생성합니다. 이것이 통신의 첫 단추입니다.

## 10.2 단순 엔드포인트 검색 프로토콜 (SEDP: Simple Endpoint Discovery Protocol)

참가자끼리 서로를 인식(SPDP 완료)했다면, 이제 구체적으로 어떤 데이터를 주고받을지(Topic)에 대한 정보를 교환해야 합니다. 이를 위해 SEDP를 사용합니다.

- **동작 방식:**
  1. SPDP를 통해 맺어진 연결을 바탕으로, 이번에는 **신뢰성 있는(Reliable)** 통신을 사용하여 정보를 교환합니다.
  2. RTPS 표준은 이 정보를 교환하기 위해 **\*\*사전 정의된(Pre-defined)** 4개의 내장 토픽(Built-in Topics)\*\*을 사용합니다.
- **내장 엔드포인트 (Built-in Endpoints):**
  1. **Publications Writer/Reader:** "나 이런 데이터를 쓸(Write) 거야"라고 알림 (Topic 이름, Type 이름, QoS 등).
  2. **Subscriptions Writer/Reader:** "나 이런 데이터를 읽고(Read) 싶어"라고 알림.

## 10.3 검색 트래픽의 내용 (Discovery Traffic)

검색 단계에서 오고 가는 데이터는 일반 데이터와 구조가 다릅니다. 확장성을 위해 **ParameterList** 형식을 사용합니다.

### 10.3.1 파라미터 리스트 (ParameterList)

- 구조: PID (Parameter ID) + Length + Value 의 연속된 구조입니다. (TLV 방식과 유사)
- 장점:
  - 새로운 QoS 정책이 추가되더라도, 이전 버전의 구현체는 모르는 PID를 건너뛰고 해석할 수 있어 하위 호환성이 보장됩니다.
  - 마지막은 항상 PID\_SENTINEL로 끝납니다.

### 10.3.2 주요 파라미터 (Key Parameters)

- PID\_TOPIC\_NAME: 데이터의 주제 이름 (예: "RobotSensor").
- PID\_TYPE\_NAME: 데이터 타입 이름 (예: "SensorDataStruct").
- PID\_RELIABILITY: 신뢰성 QoS 설정 (Reliable 또는 Best-Effort).
- PID\_DURABILITY: 지속성 QoS 설정 (Volatile 또는 Transient-Local).

---

## 10.4 매칭 및 연결 (Matching and Connection)

SEDP를 통해 상대방의 Writer/Reader 정보를 수신하면, 각 엔티티는 **매칭(Matching)** 과정을 수행합니다.

1. **토픽 일치:** Topic 이름과 Type 이름이 같은가?
2. **QoS 호환성(RxO: Requested vs Offered):**
  - Writer가 제공하는 품질(Offered)이 Reader가 요구하는 품질(Requested)보다 높거나 같아야 합니다.
  - 예: Reader가 "Reliable"을 원하는데 Writer가 "Best-Effort"만 지원하면 연결되지 않습니다.
3. **결과:** 모든 조건이 맞으면 논리적 연결이 성립되고, 이때부터 애플리케이션 데이터(User Data)가 흐르기 시작합니다.

---

## 10.5 내장 엔티티 ID (Built-in Entity Ids)

RTPS 헤더 내에서 검색 트래픽을 구분하기 위해 고정된 EntityID를 사용합니다. (매우 중요)

| Entity ID | 용도 | 설명 |
|-----------|----|----|
|-----------|----|----|

|            |                 |                   |
|------------|-----------------|-------------------|
| 0x000100c2 | SPDP Writer     | 참가자 정보 송신용        |
| 0x000100c7 | SPDP Reader     | 참가자 정보 수신용        |
| 0x000300c2 | SEDP Pub Writer | 자신의 Writer 정보 송신용 |
| 0x000300c7 | SEDP Pub Reader | 상대의 Writer 정보 수신용 |
| 0x000400c2 | SEDP Sub Writer | 자신의 Reader 정보 송신용 |
| 0x000400c7 | SEDP Sub Reader | 상대의 Reader 정보 수신용 |

(참고: 마지막 바이트 c2는 Writer, c7은 Reader를 의미하는 RTPS 관례입니다.)

## [번역자 주석] 10장의 핵심 요약

### 1. 부트스트래핑(Bootstrapping):

- SPDP는 "답이 먼저냐 알이 먼저냐" 문제를 해결합니다. 아무것도 모르는 상태에서 멀티캐스트로 "나 여기 있어!"라고 외치는 것이 시작입니다.

### 2. 시스템 트래픽 vs 사용자 트래픽:

- 10장에서 설명하는 모든 통신은 사용자가 보내는 데이터가 아니라, 미들웨어 자체가 돌아가기 위한 \*\*메타 데이터(Meta-data)\*\*입니다.
- 따라서 네트워크 패킷을 분석할 때, EntityID를 보고 이것이 '데이터'인지 '검색 정보'인지 구분해야 합니다.

### 3. QoS 매칭:

- DDS가 강력한 이유입니다. 단순히 이름만 같다고 연결하는 것이 아니라, "나는 1초에 한 번은 꼭 받아야 해(Deadline)" 같은 조건이 맞아야만 연결해 줍니다. 이 로직이 SEDP 과정에 포함되어 있습니다.